

Rule-Based Python Chatbot

Target Flow: INPUT → PROCESS → RESPOND → REPEAT → EXIT | Course: Python Programming / AI Foundations

Name: Sorna parvathi.M	Register No: 99230041128
Department: Computer Science and Engineering (CSE)	Date: 11-09-2026

1. Task Overview & Requirements Compliance

This document presents the complete implementation of a conversational rule-based chatbot built in Python, developed strictly adhering to the specified requirements. The chatbot operates in an interactive continuous execution loop, processes user inputs via normalization, handles multiple greetings and topical queries, personalizes responses, and terminates gracefully upon user request.

#	Requirement Description	Implementation Logic
1	Ask user's name before the main loop	<code>input()</code> called before <code>while True</code> loop, stored in <code>user_name</code>
2	Use <code>while True</code> : for continuous execution	Infinite while loop maintained until explicit <code>break</code>
3	Use <code>input()</code> to receive messages	Dynamic prompt: <code>input(f'{user_name}: ')</code>
4	Clean input using <code>.lower().strip()</code>	<code>raw_input.lower().strip()</code> applied immediately to normalize input
5	Recognize at least 2 greetings ('hello', 'hi')	Handles 'hello', 'hi', and 'hey' using flexible conditional checks
6	Answer at least 3 questions/topics	Covers status inquiries, capabilities/skills, and Python domain queries
7	Use <code>if / elif / else</code> decision making	Structured multi-branch sequential conditional chain
8	Use <code>break</code> to exit when typing 'bye' or 'exit'	Detects termination keywords and cleanly executes <code>break</code>
9	Use user's name in at least one response	Interpolates <code>user_name</code> in greeting, status check, and exit routine
10	Useful fallback response using <code>else</code>	Catches unrecognized input with guidance prompt without crashing

2. Python Chatbot Source Code

```
# =====
# Task: Rule-Based Conversational Chatbot
# Student: Sorna parvathi.M (Reg No: 99230041128)
# Department: CSE | Date: 11-09-2026
# =====

# 1. Ask the user's name before entering the conversation loop
user_name = input("Hello! What is your name? ").strip()
if not user_name:
    user_name = "Friend"

print(f"Nice to meet you, {user_name}! I am PyBot.")
print("You can chat with me, ask questions, or type 'bye' or 'exit' to quit.\n")

# 2. Continuous loop to sustain conversational flow
while True:
    # 3 & 4. Receive message and clean via .lower().strip()
```

```

raw_message = input(f"{user_name}: ")
clean_message = raw_message.lower().strip()

# 8. Exit handling via 'break'
if "bye" in clean_message or "exit" in clean_message:
    # 9. Use user's name in the response
    print(f"PyBot: Goodbye, {user_name}! It was wonderful chatting with you.")
    break

# 5 & Challenge: Flexible greeting recognition using the 'in' operator
elif "hello" in clean_message or "hi" in clean_message or "hey" in clean_message:
    print(f"PyBot: Hello there, {user_name}! How can I assist you today?")

# 6. Topic 1 - Well-being / Status check
elif "how are you" in clean_message or "how do you do" in clean_message:
    print(f"PyBot: I'm operating at peak performance, {user_name}! Thank you for asking.")

# 6. Topic 2 - Capabilities / Help
elif "what can you do" in clean_message or "help" in clean_message or "skills" in clean_message:
    print("PyBot: I can answer questions about programming, discuss Python fundamentals,")
    print("        or converse with you. Ask me 'what is python' to see!")

# 6. Topic 3 - Core domain knowledge: Python programming
elif "python" in clean_message:
    print("PyBot: Python is a versatile, high-level language loved for its clean syntax,")
    print("        readability, and rich library ecosystem in AI and Data Science.")

# Extra Topic 4 - Motivation / Fun
elif "joke" in clean_message or "funny" in clean_message:
    print("PyBot: Why do Python programmers wear glasses? Because they don't C#! 😊")

# 10. Fallback response for unhandled messages
else:
    print(f"PyBot: I'm not sure how to respond to that, {user_name}.")
    print("        Try asking about Python, asking for help, or saying 'hello'!")

```

3. Test Run Log and Verification

Test Case	Input Tested	Observed Chatbot Output / Behavior
Test with 'hello'	hello	PyBot: Hello there, Sorna! How can I assist you today?
Test with 'HELLO'	HELLO	PyBot: Hello there, Sorna! How can I assist you today? (Normalized via .lower())
Test with extra whitespace	hello	PyBot: Hello there, Sorna! How can I assist you today? (Handled via .strip())
Test with 'hi'	hi	PyBot: Hello there, Sorna! How can I assist you today?
Challenge test (variation)	hey hello bot	PyBot: Hello there, Sorna! (Flexible sub-string match via 'in' operator)
Test question 1	how are you	PyBot: I'm operating at peak performance, Sorna! Thank you for asking.
Test question 2 (topic)	what is python?	PyBot: Python is a versatile, high-level language loved for its clean syntax...
Test unknown message	who built the pyramids?	PyBot: I'm not sure how to respond to that, Sorna. Try asking about Python...
Exit test	bye	PyBot: Goodbye, Sorna! It was wonderful chatting with you. (Loop terminates)

4. Architectural Target Flow Breakdown

- **1. INPUT Phase:** The user enters text using `input()`. The prompt includes the user's name for a conversational, personalized experience.
- **2. PROCESS Phase:** The raw input string is normalized using `.lower()` (ensuring case insensitivity) and `.strip()` (removing leading and trailing whitespaces).
- **3. RESPOND Phase:** An `if-elif-else` conditional structure evaluates keywords. The Python `in` operator enables flexible matching for variations like 'hey hello bot'.
- **4. REPEAT Phase:** The `while True:` loop keeps the conversation running seamlessly across multiple exchanges.
- **5. EXIT Phase:** When 'bye' or 'exit' is detected, a personalized farewell is printed, and `break` terminates execution.